

Nome:

Nota:

QUADRO DE RESPOSTAS

Q1 A) B) C) D) E) F) G)

Q2 A) B) C) D) E) | Q3 | Q4 | Q5 | Q6

CENÁRIO

Considere um sistema que registra a quantidade de chuva acumulada em diferentes locais. Cada objeto da classe `MedidorChuva` representa um medidor e armazena o total de chuva registrado até o momento. Os arquivos abaixo pertencem ao mesmo projeto e ao mesmo pacote.

MedidorChuva.java

```
1 public class MedidorChuva {
2     String local;
3     double totalMilimetros;
4
5     void registrarChuva(double milimetros) {
6         totalMilimetros = totalMilimetros + milimetros;
7     }
8
9     double consultarTotal() {
10        return totalMilimetros;
11    }
12 }
```

Main.java

```
1 public class Main {
2     public static void main(String[] args) {
3         MedidorChuva principal = new MedidorChuva();
4         principal.totalMilimetros = 6.0;
5         MedidorChuva apoio = principal;
6
7         MedidorChuva outro = new MedidorChuva();
8         outro.totalMilimetros = 6.0;
9
10        System.out.println(principal == outro);
11        System.out.println(principal == apoio);
12
13        apoio = outro;
14        System.out.println(principal == apoio);
15
16        apoio.registrarChuva(3.0);
17        System.out.println(principal.consultarTotal());
18        System.out.println(outro.consultarTotal());
19
20        apoio = principal;
21        apoio.registrarChuva(5.0);
22        System.out.println(principal.consultarTotal());
23
24        apoio.totalMilimetros = -4.0;
25        principal.registrarChuva(6.0);
26        System.out.println(principal.consultarTotal());
27    }
28 }
```

QUESTÕES

Q1. (35 pontos) Considere a execução completa da classe `Main` e indique o valor exibido no console em cada uma das impressões abaixo.

A) Linha 10

B) Linha 11

C) Linha 14

D) Linha 17

E) Linha 18

F) Linha 22

G) Linha 26

Q2. (25 pontos) Analise as afirmativas abaixo e assinale (V) para as verdadeiras e (F) para as falsas.

A) Na classe `MedidorChuva`, os atributos `local` e `totalMilimetros` representam comportamentos, enquanto os métodos `registrarChuva` e `consultarTotal` representam o estado do objeto.

B) Depois que `registrarChuva` termina, o parâmetro `milimetros` permanece armazenado no objeto como se fosse um atributo.

C) Como `registrarChuva` possui retorno `void`, ele não pode alterar os dados armazenados no objeto.

D) Se uma variável receber o valor retornado por `consultarTotal()`, alterar essa variável depois também modifica `totalMilimetros` no objeto.

E) Um método pode consultar ou alterar um atributo do próprio objeto sem receber esse atributo como parâmetro.

QUESTÕES – CONTINUAÇÃO

Q3. (10 pontos) Agora queremos que cada **MedidorChuva** possa informar se o total de chuva acumulada atingiu um determinado limite.

Qual das propostas abaixo permite que o próprio objeto faça essa verificação usando o valor de **totalMilimetros** que ele já armazena?

A) Adicionar um atributo para guardar se o medidor está em alerta:

```
boolean emAlerta;
```

B) Fazer a comparação diretamente na classe **Main**:

```
boolean alerta = medidor.consultarTotal() >= limite;
```

C) Adicionar um método ao **MedidorChuva**, mas exigir que o total também seja informado:

```
boolean atingiuAlerta(double total, double limite) {  
    return total >= limite;  
}
```

D) Permitir que outros trechos do programa acessem diretamente o total para fazer a comparação:

```
public double totalMilimetros;
```

E) Adicionar um método que compara o limite recebido com o total armazenado no próprio objeto:

```
boolean atingiuAlerta(double limite) {  
    return totalMilimetros >= limite;  
}
```

Q4. (10 pontos) Na versão inicial de **MedidorChuva**, o atributo **totalMilimetros** foi declarado sem um modificador de acesso. Mesmo assim, a classe **Main** consegue executar:

```
principal.totalMilimetros = 6.0;
```

Qual alternativa explica corretamente por que esse acesso é permitido?

A) Porque o método **main** possui permissão especial para acessar os atributos de qualquer classe.

B) Porque um atributo declarado sem modificador de acesso pode ser acessado por outras classes do mesmo pacote.

C) Porque todo atributo do tipo **double** pode ser acessado diretamente por qualquer classe do projeto.

D) Porque uma variável que guarda a referência de um objeto pode acessar qualquer atributo desse objeto, independentemente do modificador de acesso.

E) Porque a criação do objeto com **new MedidorChuva()** torna seus atributos públicos.

Q5. (10 pontos) Considere agora uma nova versão da classe **MedidorChuva** em que:

- **totalMilimetros** é **private**;
- **registrarChuva** soma apenas valores maiores que zero;

Considere o seguinte início:

```
MedidorChuva principal = new MedidorChuva();  
MedidorChuva apoio = principal;
```

Qual alternativa mantém **principal** e **apoio** apontando para o mesmo objeto e, ao final, faz **consultarTotal()** devolver **12.0** pelas duas referências?

A)

```
principal.registrarChuva(7.0);  
apoio = new MedidorChuva();  
apoio.registrarChuva(5.0);
```

B)

```
principal.totalMilimetros = 7.0;  
apoio.totalMilimetros = -3.0;  
apoio.totalMilimetros = 12.0;
```

C)

```
principal.registrarChuva(7.0);  
apoio.registrarChuva(3.0);  
apoio.registrarChuva(5.0);
```

D)

```
principal.registrarChuva(7.0);  
apoio.registrarChuva(-3.0);  
apoio.registrarChuva(5.0);
```

E)

```
principal.registrarChuva(7.0);  
apoio.consultarTotal();  
apoio.registrarChuva(-3.0);
```

Q6. (10 pontos) Agora **totalMilimetros** é **private**, e seu valor só deve ser alterado por meio de **registrarChuva**, que aceita apenas valores maiores que zero.

Considere a proposta de adicionar o seguinte método à classe:

```
public void definirTotal(double novoTotal) {  
    totalMilimetros = novoTotal;  
}
```

Essa mudança mantém o controle sobre as alterações de **totalMilimetros**?

A) Sim. Como **definirTotal** pertence à própria classe, qualquer alteração realizada por ele respeita automaticamente as regras definidas para **MedidorChuva**.

B) Não. Um método **public** não pode alterar um atributo **private** da própria classe.

C) Não. O método **definirTotal** permite substituir diretamente o valor de **totalMilimetros**, sem passar pela validação feita em **registrarChuva**.

D) Sim. Como **totalMilimetros** é **private**, nenhum método público pode atribuir diretamente um novo valor ao atributo.

E) Não. Ao receber **novoTotal** como parâmetro, **totalMilimetros** deixa de ser **private**.